

XÂY DỰNG CATSA HOÀN CHỈNH

Hướng dẫn từng bước, có ví dụ minh họa

Tích hợp ba thành phần:

Module 1 — Category-Enhanced Session Graph

Module 2 — Category-Structure-Guided Augmentation

Session-level InfoNCE Loss

Người hướng dẫn: PGS. TS. Nguyễn Văn Hậu

Tháng 5/2026

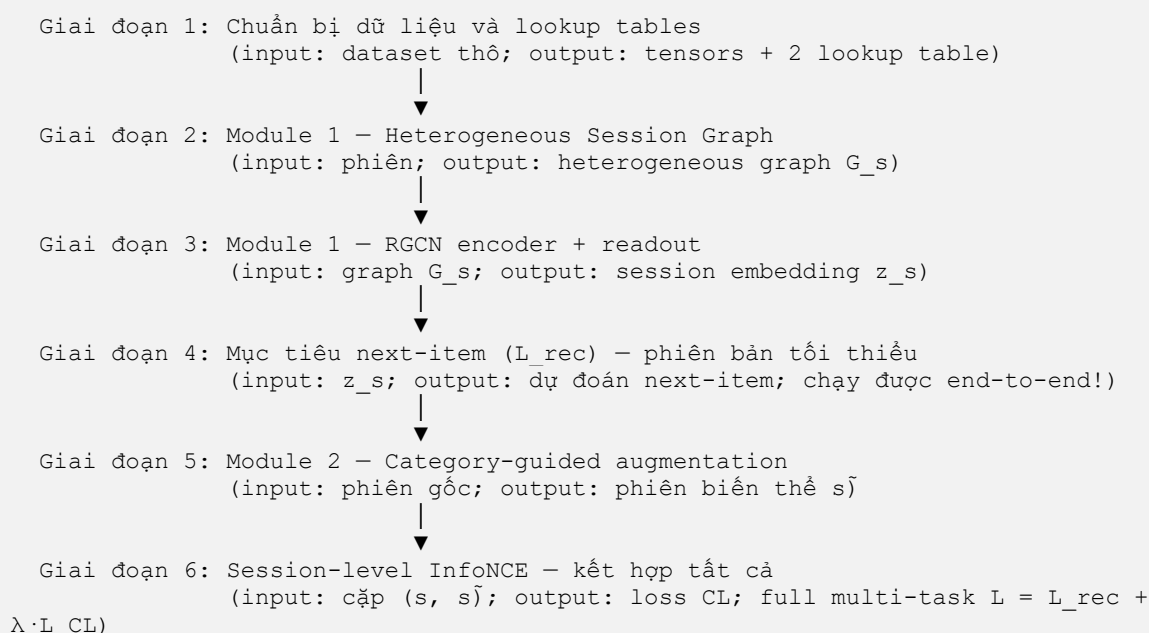
LỜI MỞ ĐẦU

Tài liệu này là bước tiếp theo sau hai tài liệu trước — Background Motivation Q3 (giải thích trực quan các khái niệm) và Hướng dẫn Thực nghiệm Q3 (giải thích cách so sánh baselines). Hai tài liệu trước trả lời câu hỏi WHAT (CatSA là gì) và WHY (vì sao thiết kế thế). Tài liệu này trả lời câu hỏi HOW — em xây dựng CatSA cụ thể như thế nào, từng bước một.

Cách tiếp cận của tôi là chia việc xây dựng CatSA thành SÁU GIAI ĐOẠN kế tiếp nhau theo nguyên tắc scaffolding. Mỗi giai đoạn xây trên giai đoạn trước, có deliverable cụ thể, có sanity check để em xác nhận đã làm đúng trước khi sang giai đoạn sau. Cách này tránh được tình huống mà tôi đã thấy nhiều NCS gặp phải: ngồi viết hết toàn bộ code 2000 dòng rồi mới chạy, thấy không hoạt động, không biết bug ở đâu, phải debug lại từ đầu rất tốn thời gian. Với scaffolding, em luôn có một version đang chạy ổn định ở trạng thái nào đó, và mỗi lần thêm một thành phần mới em chỉ cần debug phần đó.

Sáu giai đoạn được sắp xếp theo logic phụ thuộc dữ liệu — em không thể làm Module 2 (augmentation) trước khi có lookup table category-to-items, không thể làm session-level CL trước khi có encoder để sinh embedding phiên. Sau đây là tổng quan sáu giai đoạn:

TỔNG QUAN SÁU GIAI ĐOẠN — XÂY DỰNG CATSA:



Nguyên tắc cốt lõi: tại cuối Giai đoạn 4, em đã có một mô hình end-to-end CHẠY ĐƯỢC — đó là một version của "SR-GNN với heterogeneous category graph". Mô hình này có thể train, có thể eval, có thể cho kết quả HR@20. Đây là version A2 trong bảng ablation (Module 1 only, không có Module 2 và CL). Sau Giai đoạn 4, em đã có MỘT KẾT QUẢ ĐỂ BÁO CÁO. Giai đoạn 5 và 6 thêm Module 2 và CL để được CatSA đầy đủ. Nếu Giai đoạn 5 hoặc 6 gặp vấn đề lớn, em vẫn không trắng tay — A2 đã có sẵn.

Ngân sách thời gian dự kiến cho sáu giai đoạn: tổng cộng 6-8 tuần (tháng T4 và phần đầu T5 trong lộ trình), không tính thời gian chạy thí nghiệm cuối cùng. Cụ thể: Giai đoạn 1 mất 1 tuần, Giai đoạn 2 mất 1 tuần, Giai đoạn 3 mất 1.5 tuần, Giai đoạn 4 mất 1 tuần, Giai đoạn 5 mất 1.5 tuần, Giai đoạn 6 mất 1 tuần. Em báo Thầy ngay tại các check-point nếu gặp khó khăn — đừng cố một mình lâu hơn 3 ngày.

Trước khi bắt đầu — chuẩn bị môi trường

Em cần đảm bảo môi trường code đã sẵn sàng. Cụ thể: Python 3.9 hoặc 3.10 (tránh 3.11+ vì một số thư viện chưa support tốt), PyTorch 2.0+ với CUDA support, PyTorch Geometric (PyG) 2.4+ hoặc DGL 1.1+ — tôi khuyến nghị PyG vì cộng đồng SBR dùng phổ biến hơn. Em đã có các thư viện này từ giai đoạn reproduce baseline, nên chỉ cần check version.

Một số thói quen tốt em nên thiết lập từ đầu: dùng Git để commit mỗi khi qua một check-point (em sẽ thấy giá trị khi cần rollback); viết unit test cho từng module nhỏ trước khi tích hợp; dùng configuration file (YAML hoặc JSON) thay vì hard-code hyperparameter trong code; log mọi thí nghiệm qua TensorBoard hoặc Weights & Biases. Mất 30 phút setup ban đầu nhưng tiết kiệm hàng giờ debug về sau.

GIAI ĐOẠN 1 — CHUẨN BỊ DỮ LIỆU VÀ LOOKUP TABLES

Mục tiêu giai đoạn này: chuyển dataset thô (Yoochoose, Diginetica, RetailRocket, Tmall) thành các cấu trúc dữ liệu mà CatSA có thể đọc nhanh trong training. Tốc độ là quan trọng — pipeline xử lý dữ liệu chậm sẽ làm bottleneck cho toàn bộ thí nghiệm.

Em đã có một phần kết quả từ giai đoạn reproduce baseline (T1-T2): file train.txt, val.txt, test.txt với mỗi dòng là một phiên dạng "item_1 item_2 ... item_n". Trong giai đoạn này em làm thêm hai việc mới: (1) gán category cho mỗi item, (2) xây hai lookup table phục vụ Module 2.

1.1. Cấu trúc dữ liệu cần có sau Giai đoạn 1

CẤU TRÚC DỮ LIỆU SAU GIAI ĐOẠN 1:

(1) Dictionary item2cat — gán mỗi item vào leaf category:

```
item2cat = {
    1234: "Điện thoại",      # iPhone 16 Pro
    1235: "Ốp lưng",        # Ốp lưng iPhone
    1236: "Sạc & Cáp",      # Sạc 20W
    1237: "Tai nghe",        # AirPods Pro 2
    ...
}
```

Kích thước: $|I|$ (số item trong vocabulary)

(2) Dictionary cat2items — lookup ngược (cho Module 2 same-leaf):

```
cat2items = {
    "Điện thoại": [1234, 5678, 9012, ...], # mọi item thuộc cat
    "Ốp lưng":    [1235, 3456, 7890, ...],
    ...
}
```

Kích thước: $|C| \times$ danh sách items

Trong Module 2 same-leaf, em tra cat2items để chọn item thay thế.

(3) Dictionary cat_parent — chỉ cần khi có taxonomy (RetailRocket, Tmall):

```
cat_parent = {
    "Điện thoại": "Điện thoại & Phụ kiện",
    "Ốp lưng":    "Điện thoại & Phụ kiện",
    "Tai nghe":   "Điện thoại & Phụ kiện",
    "Nồi cơm điện": "Nhà bếp",
    ...
}
```

Trong Module 2 sibling-leaf, em tra cat_parent → tìm siblings → chọn.

(4) Sessions — đã có từ giai đoạn reproduce baseline:

```
train_sessions: list of [item_1, ..., item_n]
val_sessions:   list of [item_1, ..., item_n]
test_sessions:  list of [item_1, ..., item_n]
```

1.2. Hướng dẫn cụ thể cho từng dataset

Mỗi dataset có format khác nhau, nên cách đọc category cũng khác. Tôi mô tả ngắn gọn cho từng dataset; em sẽ thấy việc này tương đối routine.

Đối với Yoochoose 1/64, file gốc yoochoose-clicks.dat có 4 cột: SessionID, Timestamp, ItemID, Category. Cột Category là số nguyên đại diện cho leaf category (flat). Em đọc file, group by SessionID

để có phiên, và đồng thời build item2cat từ cột ItemID và Category. Yoochoose không có taxonomy nên cat_parent không cần.

Đối với Diginetica, dataset có file train-item-views.csv (sessions) và products.csv (mapping item → category). Em load products.csv để build item2cat. Cũng chỉ có flat category. Lưu ý đặc thù của Diginetica: category đã được hash thành số nguyên ngẫu nhiên (ẩn danh hóa), nên không có ý nghĩa ngữ nghĩa rõ — nhưng vẫn dùng được làm tín hiệu category.

Đối với RetailRocket — đây là dataset quan trọng nhất vì có taxonomy đầy đủ. RetailRocket có 3 file: events.csv (interactions), item_properties.csv (item metadata bao gồm category), category_tree.csv (cây taxonomy đầy đủ với parent-child relations). Em build item2cat từ item_properties (lọc property = 'categoryid'), build cat_parent từ category_tree.csv. Đây là dataset nơi sibling-leaf substitution phát huy hết giá trị.

Đối với Tmall (IJCAI-15), dataset có file user_log.csv với cột cat_id (leaf category) và brand_id, seller_id (metadata phụ). Tmall có cấu trúc taxonomy 3 cấp nhưng không có file riêng cho cây — em phải infer từ data. Trong tài liệu này, để giữ phạm vi rõ, em chỉ dùng leaf category cho Tmall (giống flat); phần taxonomy mở rộng để dành cho bài Q1.

1.3. Ví dụ trực quan — xây cat2items cho dataset nhỏ

Để em hình dung cụ thể, hãy tưởng tượng một dataset đồ chơi với 9 item, 4 category. Em build cat2items như sau:

VÍ DỤ MINH HOẠ — DATASET ĐỒ CHƠI:

```
item2cat = {
    1: "Điện thoại",      2: "Điện thoại",      3: "Điện thoại",
    4: "Ôp lung",         5: "Ôp lung",
    6: "Sạc & Cáp",       7: "Sạc & Cáp",
    8: "Tai nghe",        9: "Tai nghe"
}
```

Sau khi đảo chiều (cat2items):

```
cat2items = {
    "Điện thoại": [1, 2, 3],
    "Ôp lung":    [4, 5],
    "Sạc & Cáp":  [6, 7],
    "Tai nghe":   [8, 9]
}
```

Cách build trong Python (pseudocode):

```
from collections import defaultdict
cat2items = defaultdict(list)
for item_id, cat in item2cat.items():
    cat2items[cat].append(item_id)
cat2items = dict(cat2items) # convert back to regular dict
```

Khi cần substitute item 1 (Điện thoại) trong Module 2 same-leaf:

```
candidates = cat2items["Điện thoại"] # [1, 2, 3]
candidates_excluding_self = [c for c in candidates if c != 1] # [2, 3]
replacement = random.choice(candidates_excluding_self) # 2 hoặc 3
```

1.4. Lưu ý kỹ thuật quan trọng

Có ba lưu ý kỹ thuật cần chú ý ở giai đoạn này, nếu bỏ sót sẽ gây lỗi khó debug ở các giai đoạn sau.

Lưu ý thứ nhất là về index hóa item. Trong dataset thô, item ID có thể là số lớn (như 214716935 trên Yoochoose) hoặc string. Khi đưa vào PyTorch, em phải remap thành số nguyên liên tiếp 0, 1, 2, ..., $|I|-1$ (đây là index cho embedding lookup). Em build một mapping $\text{item_id_raw} \rightarrow \text{item_idx}$ và lưu lại file để convert qua lại. KHÔNG dùng trực tiếp item_id_raw làm index.

Lưu ý thứ hai là về index hóa category. Tương tự item, em remap category labels thành số nguyên 0, 1, ..., $|C|-1$. Sau đó item2cat trở thành $\text{item2cat}[\text{item_idx}] = \text{cat_idx}$ (trả về số nguyên thay vì string). Điều này giúp tốc độ lookup nhanh và dễ tích hợp với PyTorch tensor.

Lưu ý thứ ba và quan trọng nhất là về data leakage. Khi build cat2items và cat_parent , em CHỈ dùng item xuất hiện trong train + validation set, KHÔNG dùng item chỉ có trong test set. Lý do: cold-start protocol (20% item chỉ xuất hiện trong test) yêu cầu mô hình không được "biết" các cold item này khi training. Nếu cat2items chứa cả cold item, mô hình có thể "thấy" chúng qua substitution — đây là leakage. Bài Q3 không nhắm cold-start, nhưng vẫn nên tuân thủ protocol này để giữ pipeline thống nhất với bài Q2.

1.5. Sanity check cuối Giai đoạn 1

Trước khi sang Giai đoạn 2, em chạy bốn kiểm tra đơn giản để xác nhận dữ liệu chuẩn bị đúng:

```
CHECK 1 – Mọi item trong train có category:
    assert all(item in item2cat for item in train_items)

CHECK 2 – cat2items đảo chiều đúng:
    for cat, items in cat2items.items():
        for item in items:
            assert item2cat[item] == cat    # consistency

CHECK 3 – Phân bố category size không quá lệch:
    sizes = [len(items) for items in cat2items.values()]
    print(f"|C| = {len(cat2items)}")
    print(f"min size: {min(sizes)}, max size: {max(sizes)}")
    print(f"median size: {sorted(sizes)[len(sizes)//2]}")
    print(f"#cat with size < 5: {sum(1 for s in sizes if s < 5)}")

    → Nếu nhiều cat có size < 5, em đã lường trước
       bằng cơ chế fallback  $k_{\min}$  trong Module 2 (Giai đoạn 5).

CHECK 4 – Taxonomy nhất quán (chỉ với RetailRocket):
    for child, parent in cat_parent.items():
        assert child != parent    # không tự tham chiếu
        # parent có trong cat_parent hoặc là root
        assert parent in cat_parent or parent == "ROOT"
```

Gate Check Giai đoạn 1: (a) Có item2cat , cat2items đầy đủ cho cả 4 dataset. (b) Có cat_parent cho RetailRocket. (c) Bốn check trên pass cho mỗi dataset. (d) Tất cả file đã save dưới dạng pickle

hoặc JSON, có thể load lại nhanh. Khi pass tất cả, em commit Git với message "Stage 1 complete" và tiếp Giai đoạn 2.

GIAI ĐOẠN 2 — MODULE 1, PHẦN A: HETEROGENEOUS SESSION GRAPH

Mục tiêu giai đoạn này là viết hàm `session_to_graph` nhận đầu vào là một phiên (danh sách item index) và trả về một heterogeneous graph chứa cả item nodes và category nodes, cùng các loại cạnh khác nhau. Đây là PHẦN A của Module 1 — phần xây cấu trúc graph. PHẦN B (RGCN encoder) sẽ làm ở Giai đoạn 3.

Vì sao tách Module 1 thành hai giai đoạn? Bởi vì xây graph và xây encoder là hai vấn đề khác nhau, có thể debug độc lập. Khi em hoàn thành Giai đoạn 2, em có thể kiểm tra graph được xây có đúng không (bằng visualize hoặc print) trước khi đi vào logic RGCN phức tạp hơn ở Giai đoạn 3.

2.1. Cấu trúc heterogeneous graph cần xây

Đã giải thích chi tiết trong tài liệu walkthrough Phần 5.4 và Phần 3.4. Tôi nhắc lại ngắn gọn cấu trúc rồi đi vào implementation.

CẤU TRÚC HETEROGENEOUS GRAPH $G_s = (V, E)$:

Nodes V có ba loại:

$V_{\text{item}} = \{i_1, \dots, i_n\}$	(các item trong phiên)
$V_{\text{cat}} = \{\text{cat}(i_j) : j = 1..n\}$	(leaf categories)
$V_{\text{parent}} = \{\text{parent}(c) : c \in V_{\text{cat}}\}$	(chỉ khi có taxonomy)

Edges E có ba loại quan hệ:

sequential: $\{(i_j, i_{j+1}) : j = 1..n-1\}$	(transition trong phiên)
membership: $\{(i_j, \text{cat}(i_j)) : j = 1..n\}$	(item $\rightarrow$ category)
taxonomy: $\{(c, \text{parent}(c)) : c \in V_{\text{cat}}\}$	(chỉ khi có taxonomy)

Tính chất quan trọng: graph này LOCAL — riêng cho mỗi phiên.
Mỗi phiên có graph riêng, không chia sẻ giữa các phiên trong batch.
Khi train, mỗi batch có B phiên $\rightarrow B$ graphs riêng biệt.

2.2. Implementation với PyTorch Geometric

PyG có một class chuyên dụng cho heterogeneous graph là `HeteroData`. Em dùng class này để biểu diễn G_s . Logic implementation gồm bốn bước theo thứ tự.

Bước thứ nhất là gom các unique items, categories, và parents trong phiên. Vì một item có thể xuất hiện nhiều lần trong phiên (ví dụ user click iPhone hai lần), em chỉ giữ unique để tránh duplicate node. Tuy nhiên, sequential edges vẫn được tạo theo thứ tự nguyên thủy (kể cả khi node đã xuất hiện trước).

Bước thứ hai là tạo node features. Mỗi loại node cần có feature ban đầu để GNN có thể aggregate. Cho item nodes, feature là item embedding (lookup từ một embedding table $|I| \times d$). Cho category nodes, feature là category embedding (lookup từ một bảng riêng $|C| \times d$). Cho parent nodes, dùng cùng category embedding table — không cần bảng riêng cho parent.

Bước thứ ba là tạo edge index cho từng loại quan hệ. Mỗi loại edge được lưu thành một tensor $2 \times$ (số edges của loại đó), với hàng đầu là source node, hàng thứ hai là target node. PyG yêu cầu format này để message passing hoạt động đúng.

Bước thứ tư là gói tất cả vào `HeteroData` object. Đây là input cho RGCN encoder ở Giai đoạn 3.

2.3. Ví dụ minh hoạ — phiên ví dụ thành graph

Hãy xây graph cho phiên ví dụ chuẩn của luận án để em hình dung cụ thể. Giả sử em đang xét RetailRocket (có taxonomy).

```
PHIÊN GỐC:
s = [iPhone 16 Pro, Ốp lưng iPhone, Sạc 20W, AirPods Pro 2]

Sau khi index hoá:
s_idx = [1234, 1235, 1236, 1237]

Categories:
cat(1234) = "Điện thoại"           → cat_idx = 7
cat(1235) = "Ốp lưng"             → cat_idx = 8
cat(1236) = "Sạc & Cáp"           → cat_idx = 9
cat(1237) = "Tai nghe"            → cat_idx = 10

Parents (từ cat_parent dictionary):
parent("Điện thoại") = "Điện thoại & Phụ kiện" → parent_idx = 50
parent("Ốp lưng")    = "Điện thoại & Phụ kiện" → parent_idx = 50
parent("Sạc & Cáp")  = "Điện thoại & Phụ kiện" → parent_idx = 50
parent("Tai nghe")   = "Điện thoại & Phụ kiện" → parent_idx = 50
(cả 4 cùng parent — phiên này có intent rất nhất quán)

Nodes của graph:
V_item   = [1234, 1235, 1236, 1237]   (4 nodes)
V_cat    = [7, 8, 9, 10]              (4 nodes — unique)
V_parent = [50]                       (1 node — unique)

Edges:
sequential: (1234→1235), (1235→1236), (1236→1237)   (3 edges)
membership: (1234→7), (1235→8), (1236→9), (1237→10) (4 edges)
taxonomy:   (7→50), (8→50), (9→50), (10→50)          (4 edges)

Tổng: 9 nodes, 11 edges, 3 loại edge.
```

Em có thể visualize graph này bằng networkx + matplotlib để tận mắt thấy cấu trúc — đây là một sanity check rất hữu ích. Nếu em vẽ thấy 4 item nodes nối với 4 cat nodes (membership) và 4 cat nodes nối với 1 parent node (taxonomy), em đã build đúng. Mỗi sinh viên debug Module 1 lần đầu nên làm bước visualize này — phát hiện bug rất nhanh.

2.4. Một số quyết định thiết kế cần làm sớm

Có ba quyết định thiết kế ảnh hưởng đến hiệu năng và độ phức tạp của Module 1, em cần làm sớm và document trong code:

Quyết định thứ nhất là về directed vs undirected edges. Sequential edges nên là directed ($i_j \rightarrow i_{j+1}$) có nghĩa khác $i_{j+1} \rightarrow i_j$) hay undirected? Khuyến nghị: dùng directed cho sequential (giữ thông tin thứ tự), nhưng tự thêm reverse edges để message passing hai chiều. Code-wise: em có thể dùng `T.AddSelfLoops()` và `T.ToUndirected()` từ PyG transforms.

Quyết định thứ hai là về self-loops. Nên thêm self-loop cho mỗi node hay không? Khuyến nghị: có thêm self-loop cho item nodes và category nodes — giúp GNN giữ thông tin của node đó trong khi vẫn

aggregate từ neighbors. Self-loop thường được implement qua $W_{self}^{(l)}$ trong RGCN (xem Giai đoạn 3).

Quyết định thứ ba là về multi-hop taxonomy. Một số dataset (RetailRocket) có taxonomy 4-5 cấp. Em chỉ đưa parent (level liền trên) vào graph, hay đưa cả grandparent (level 2 trên), great-grandparent? Khuyến nghị cho bài Q3: chỉ dùng parent (1 cấp lên). Lý do: giữ scope đơn giản, đỉnh nhanh hơn, đủ tốt cho hầu hết tình huống. Multi-level taxonomy là scope của bài Q1.

2.5. Sanity check cuối Giai đoạn 2

```
CHECK 1 – Graph có đúng số node:
    Với phiên có n unique items và k unique categories và p unique parents:
    assert graph['item'].num_nodes == n
    assert graph['category'].num_nodes == k
    # Nếu có taxonomy:
    assert graph['parent'].num_nodes == p

CHECK 2 – Graph có đúng số edge:
    assert graph['item', 'sequential', 'item'].edge_index.size(1) == n - 1
    assert graph['item', 'membership', 'category'].edge_index.size(1) == n
    # Nếu có taxonomy:
    assert graph['category', 'taxonomy', 'parent'].edge_index.size(1) == k

CHECK 3 – Edge index không out of range:
    Mọi value trong edge_index phải < num_nodes của loại tương ứng.

CHECK 4 – Visualize 1-2 graph thực tế:
    Lấy một phiên ngẫu nhiên từ training set, build graph, vẽ với
    networkx. Kiểm tra trực quan: có đúng 3 loại edge?
    Item nodes có nối đến category nodes của chúng?

CHECK 5 – Speed test:
    Build graph cho 1000 phiên, đo thời gian.
    Mục tiêu: < 1 giây cho 1000 phiên (1 ms/phiên).
    Nếu chậm hơn, có vấn đề về implementation – báo Thầy.
```

Gate Check Giai đoạn 2: (a) Hàm `session_to_graph` nhận phiên, trả về HeteroData object. (b) Năm check trên pass. (c) Em có thể vẽ ít nhất một graph thực tế và xác nhận cấu trúc đúng. (d) Speed test pass (< 1 ms/phiên). Khi pass, commit Git với message "Stage 2 complete: graph construction".

GIAI ĐOẠN 3 — MODULE 1, PHẦN B: RGCN ENCODER + READOUT

Mục tiêu giai đoạn này: viết encoder f nhận đầu vào là heterogeneous graph G_s (từ Giai đoạn 2) và trả về session embedding z_s . Đây là phần "thông minh" của Module 1 — nơi mô hình học được các pattern trong phiên qua message passing.

Encoder f gồm hai phần lồng nhau. Phần thứ nhất là L lớp message passing kiểu RGCN (Relational Graph Convolutional Network) — mỗi lớp cập nhật node embedding bằng cách aggregate từ neighbors qua các loại quan hệ khác nhau. Phần thứ hai là soft-attention readout — gom L node embeddings của các item trong phiên thành một session embedding duy nhất z_s .

3.1. Lý thuyết RGCN trong 5 phút

RGCN (Schlichtkrull et al., ESWC 2018) mở rộng GCN cho graph có nhiều loại quan hệ. Công thức cập nhật tại lớp $l+1$ cho node v như sau:

CÔNG THỨC RGCN:

$$h_v^{(l+1)} = \sigma \left(\sum_r \sum_{u \in N_r(v)} \left(\frac{1}{c_{\{v,r\}}} \right) \cdot W_r^{(l)} \cdot h_u^{(l)} + W_{\text{self}}^{(l)} \cdot h_v^{(l)} \right)$$

Trong đó:

- | | |
|-------------------------|---|
| $r \in R$ | – tập các loại quan hệ (sequential, membership, taxonomy) |
| $N_r(v)$ | – các neighbor của v qua quan hệ r |
| $c_{\{v,r\}}$ | – hằng số chuẩn hoá, thường = $ N_r(v) $ |
| $W_r^{(l)}$ | – ma trận trọng số RIÊNG cho quan hệ r ở lớp l ($d \times d$) |
| $W_{\text{self}}^{(l)}$ | – ma trận trọng số cho self-connection |
| σ | – hàm kích hoạt phi tuyến (ReLU hoặc GELU) |

Ý nghĩa trực quan:

- Với mỗi loại quan hệ r , RGCN có một bộ trọng số W_r riêng
- Message từ neighbor qua quan hệ r được biến đổi bằng W_r
- Tất cả messages từ mọi loại quan hệ được aggregate (cộng lại)
- Cộng thêm self-connection để giữ thông tin của node đó

Khác biệt với GCN thông thường (chỉ 1 loại quan hệ):

GCN có 1 ma trận W ; RGCN có $|R|$ ma trận, mỗi ma trận học pattern của một loại quan hệ riêng biệt.

Với CatSA, ba loại quan hệ là sequential, membership, taxonomy. Em có ba ma trận W_{seq} , W_{mem} , W_{tax} , mỗi ma trận học cách aggregate message theo loại quan hệ đó. W_{seq} học pattern "item này thường xuất hiện trước/sau item kia". W_{mem} học pattern "item này thuộc category nào". W_{tax} học pattern "category này thuộc parent nào". Sau L lớp, embedding của mỗi item đã "hấp thụ" thông tin từ cả ba chiều này.

Số lớp L thường nhỏ, từ 1 đến 3. Lý do: GNN bị over-smoothing khi L lớn (mọi node embedding trở nên giống nhau). Khuyến nghị: bắt đầu với $L = 2$, tinh chỉnh sau.

3.2. Implementation RGCN với PyG

PyG có module RGCNConv sẵn — em có thể dùng trực tiếp. Tuy nhiên, RGCNConv của PyG là cho graph có node CÙNG LOẠI nhưng nhiều loại edge. Trong CatSA, em có nhiều loại NODE (item, category, parent) — nên cần kết hợp RGCNConv với HeteroConv hoặc HGTConv.

Cách đơn giản nhất: dùng HeteroConv wrapper với RGCNConv bên trong. HeteroConv sẽ tự xử lý phần multi-type node và quản lý các edge khác nhau.

PSEUDOCODE — RGCN cho heterogeneous graph:

```
from torch_geometric.nn import HeteroConv, RGCNConv

class CatSAEncoder(nn.Module):
    def __init__(self, n_items, n_cats, n_parents, d, n_layers=2):
        super().__init__()

        # Embedding tables
        self.item_emb = nn.Embedding(n_items, d)
        self.cat_emb = nn.Embedding(n_cats, d)
        # Parent dùng chung embedding với category
        # (parent là category cấp cao hơn)

        # L lớp message passing
        self.convs = nn.ModuleList()
        for _ in range(n_layers):
            conv = HeteroConv({
                ('item', 'sequential', 'item'): GCNConv(d, d),
                ('item', 'membership', 'category'): GCNConv(d, d),
                ('category', 'taxonomy', 'parent'): GCNConv(d, d),
            }, aggr='sum')
            self.convs.append(conv)

        # Soft-attention readout (theo SR-GNN)
        self.W1 = nn.Linear(d, d)
        self.W2 = nn.Linear(d, d)
        self.q = nn.Linear(d, 1)

    def forward(self, graph):
        # graph là HeteroData từ Giai đoạn 2

        # Khởi tạo node features
        x_dict = {
            'item': self.item_emb(graph['item'].node_id),
            'category': self.cat_emb(graph['category'].node_id),
            'parent': self.cat_emb(graph['parent'].node_id),
        }
        edge_index_dict = graph.edge_index_dict

        # Message passing L lớp
        for conv in self.convs:
            x_dict = conv(x_dict, edge_index_dict)
            x_dict = {k: F.relu(v) for k, v in x_dict.items()}

        # Lấy item embeddings sau cùng
        h_items = x_dict['item'] # shape: (n, d)

        # Soft-attention readout (theo SR-GNN [Wu et al. 2019])
        h_last = h_items[-1] # embedding của item cuối phiên
        alpha = self.q(torch.tanh(self.W1(h_items) + self.W2(h_last)))
        alpha = F.softmax(alpha, dim=0)
        z_s = (alpha * h_items).sum(dim=0) # shape: (d,)
```

```
return z_s    # session embedding
```

Pseudocode trên là khung làm việc. Em sẽ phải tinh chỉnh tùy theo version PyG cụ thể (API có thể thay đổi giữa các version). Tài liệu chính thức của PyG về HeteroConv khá tốt, em theo đó.

3.3. Soft-attention readout — vì sao như vậy?

Sau L lớp RGCN, em có embedding cho mọi node — bao gồm n item nodes của phiên. Câu hỏi: làm sao tổng hợp n vector này thành 1 vector z_s đại diện cho cả phiên? Có nhiều cách: lấy trung bình (mean pooling), lấy item cuối (last-item pooling), hoặc soft-attention.

SR-GNN dùng soft-attention vì lý do: trong một phiên, không phải mọi item đều quan trọng như nhau cho việc dự đoán next-item. Item cuối thường quan trọng nhất (gợi ý ngay sau click cuối), nhưng các item trước đó cũng cung cấp ngữ cảnh. Soft-attention tự học cách "cân nhắc" các item: gán weight α_j cho mỗi item, weight cao cho item quan trọng, thấp cho item ít quan trọng.

Công thức soft-attention chuẩn của SR-GNN: $\alpha_j = \text{softmax}(q^T \tanh(W_1 h_j + W_2 h_n + b))$, với h_j là embedding item j , h_n là embedding item cuối, W_1 , W_2 , q là trọng số học. Sau đó $z_s = \sum \alpha_j \cdot h_j$. Cách này được sử dụng rộng rãi trong các phương pháp SBR sau SR-GNN, nên em giữ nguyên — không cần phát minh lại.

3.4. Quan trọng — chỉ readout từ item nodes, không từ category

Đây là điểm quan trọng dễ bị bỏ sót. Sau message passing L lớp, em có embedding cho cả item nodes, category nodes, và parent nodes. Khi tổng hợp thành z_s , em CHỈ dùng item embeddings, KHÔNG dùng category hay parent embeddings.

Lý do: mục tiêu của z_s là dự đoán next-item, nên z_s phải nằm trong cùng không gian biểu diễn với item embeddings. Nếu em mix cả category và parent vào z_s , không gian biểu diễn bị nhiễu, làm giảm chất lượng dự đoán. Category và parent nodes là TRỢ GIÚP để cải thiện item embeddings qua message passing, không phải thành phần trực tiếp của z_s .

3.5. Sanity check cuối Giai đoạn 3

```
CHECK 1 — Forward pass chạy không lỗi:
graph = build_graph(sample_session)
z_s = encoder(graph)
assert z_s.shape == (d,)    # vector chiều d

CHECK 2 — z_s có gradient:
loss = z_s.sum()           # dummy loss
loss.backward()
# Kiểm tra: encoder.item_emb.weight.grad không phải None

CHECK 3 — Dimension consistency qua nhiều layer:
Set L = 1, 2, 3, chạy forward pass, kiểm tra z_s vẫn shape (d,)

CHECK 4 — Embedding tables được học:
Ban đầu, item_emb random. Sau 1 epoch training (Giai đoạn 4),
item_emb phải khác đáng kể so với init. Nếu không, có vấn đề
về gradient flow.
```

CHECK 5 – Speed test:

Forward pass cho batch 100 phiên, đo thời gian.

Mục tiêu: < 100ms cho batch 100. Nếu chậm hơn rất nhiều, kiểm tra:

- Có chạy trên GPU không?
- HeteroConv có efficient không?
- Batch size có hợp lý không?

Gate Check Giai đoạn 3: (a) Class CatSAEncoder hoàn chỉnh, forward pass trả về z_s đúng shape. (b) Năm check trên pass. (c) Speed test < 100ms cho batch 100. (d) Em đã visualize alpha values cho 1-2 phiên ngẫu nhiên — kiểm tra chúng có hợp lý (thường item cuối có alpha cao nhất). Khi pass, commit Git với message "Stage 3 complete: RGCN encoder + readout".

GIẢI ĐOẠN 4 — MỤC TIÊU NEXT-ITEM (L_REC) VÀ PIPELINE END-TO-END

Đây là giai đoạn quan trọng nhất về mặt tâm lý và thực tế. Tại cuối Giai đoạn 4, em đã có một mô hình end-to-end CHẠY ĐƯỢC trên dataset thật, có thể train, có thể eval, có thể cho ra HR@20. Đây là cột mốc an toàn — sau Giai đoạn 4, kể cả khi Giai đoạn 5 hay Giai đoạn 6 gặp vấn đề lớn, em vẫn không trắng tay. Mô hình ở Giai đoạn 4 chính là biến thể A2 trong bảng ablation (Module 1 only, không có Module 2 và CL).

Giai đoạn 4 không thêm thành phần mới về mặt khoa học — chỉ kết nối những gì đã có (Giai đoạn 1, 2, 3) thành một pipeline hoàn chỉnh và thêm hàm mất mát next-item prediction. Nhiệm vụ chính là engineering: data loader, training loop, evaluation pipeline, logging.

4.1. Hàm mất mát next-item L_rec

L_rec là hàm mất mát chuẩn cho next-item prediction trong SBR — cross-entropy giữa item ground-truth và phân phối dự đoán trên toàn vocabulary. Công thức:

HÀM MẤT MẤT NEXT-ITEM PREDICTION:

Cho phiên s với ground-truth next-item i_{target} :

1. Tính session embedding z_s qua Module 1 (Giai đoạn 3)
2. Tính score cho mỗi item $i \in I$:
 $\text{score}(s, i) = z_s \cdot h_i / \sqrt{d}$ (scaled dot-product)
hoặc đơn giản: $\text{score}(s, i) = z_s \cdot h_i$ (chuẩn của SBR)
3. Áp softmax để được phân phối:
 $P(i | s) = \text{softmax}(\text{score}(s, i))$
4. Cross-entropy:
 $L_{\text{rec}}(s) = -\log P(i_{\text{target}} | s)$

Trên một batch B :

$$L_{\text{rec}} = (1/|B|) \cdot \sum_{s \in B} L_{\text{rec}}(s)$$

Lưu ý quan trọng:

- h_i ở bước 2 là item embedding LẤY TỪ EMBEDDING TABLE (item_emb của encoder), KHÔNG phải embedding sau message passing. Lý do: embedding sau message passing chỉ tồn tại cho item TRONG phiên hiện tại. Để rank toàn vocabulary, ta cần embedding "static" của mọi item.
- Đây là tied-weight setup: cùng item embedding table được dùng cho cả input encoder và output ranking. Trick này giảm số tham số và thường cải thiện hiệu năng.

Implementation cụ thể trong PyTorch khá ngắn gọn. Em định nghĩa $\text{logits} = z_s @ \text{encoder.item_emb.weight.T}$ shape $(B, |I|)$, rồi áp $F.\text{cross_entropy}(\text{logits}, \text{target_idx})$. Đây là code chuẩn của SBR.

4.2. Data loader cho training

Mỗi phiên $s = [i_1, \dots, i_n]$ có thể được expand thành $n-1$ training samples kiểu next-item: $(s_1 = [i_1], \text{target} = i_2)$, $(s_2 = [i_1, i_2], \text{target} = i_3)$, ..., $(s_{\{n-1\}} = [i_1, \dots, i_{\{n-1\}}], \text{target} = i_n)$. Đây là cách chuẩn của SBR (gọi là "sliding window" hoặc "sequence augmentation" — đừng nhầm với augmentation của Module 2).

Em viết Dataset class kế thừa `torch.utils.data.Dataset`, mỗi `__getitem__(idx)` trả về một tuple (sub_session, target). Cụ thể: trong `__init__`, em duyệt qua mọi phiên trong train set, sinh ra mọi (sub_session, target) có thể. Trong `__getitem__`, em build heterogeneous graph cho sub_session đó (qua hàm `session_to_graph` của Giai đoạn 2), trả về (graph, target_idx).

Sau Dataset, em viết DataLoader với `collate_fn` riêng. Vì các graph có kích thước khác nhau, em không thể stack chúng trực tiếp như tensors. PyG có `Batch.from_data_list()` để gom các HeteroData thành một batch — em dùng hàm này trong `collate_fn`.

4.3. Training loop

PSEUDOCODE — TRAINING LOOP CƠ BẢN:

```
import torch
from torch.optim import Adam

model = CatSAEncoder(n_items, n_cats, n_parents, d=100, n_layers=2)
optimizer = Adam(model.parameters(), lr=1e-3, weight_decay=1e-5)

for epoch in range(n_epochs):
    model.train()
    total_loss = 0.0

    for batch in train_loader:
        graphs, targets = batch
        graphs = graphs.to(device)
        targets = targets.to(device)

        # Forward pass cho từng graph trong batch
        # (HeteroConv trong PyG xử lý batch trực tiếp)
        z_s_batch = model(graphs)  # shape: (B, d)

        # Tính scores trên toàn vocabulary
        item_embeddings = model.item_emb.weight  # shape: (|I|, d)
        logits = z_s_batch @ item_embeddings.T  # shape: (B, |I|)

        # Cross-entropy loss
        loss = F.cross_entropy(logits, targets)

        # Backward + update
        optimizer.zero_grad()
        loss.backward()
        torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=5.0)
        optimizer.step()

        total_loss += loss.item() * targets.size(0)

    avg_loss = total_loss / len(train_loader.dataset)
    print(f"Epoch {epoch}: avg train loss = {avg_loss:.4f}")

    # Validation mỗi 1 epoch
    if epoch % 1 == 0:
        hr20, ndcg20 = evaluate(model, val_loader)
        print(f"  Val HR@20 = {hr20:.4f}, NDCG@20 = {ndcg20:.4f}")

        # Early stopping nếu val không cải thiện trong 5 epochs
        if hr20 > best_hr20:
            best_hr20 = hr20
```



```

        patience = 0
        torch.save(model.state_dict(), 'best_model.pt')
    else:
        patience += 1
        if patience >= 5:
            print("Early stopping")
            break

```

4.4. Evaluation pipeline

Em đã có module `evaluation.py` từ giai đoạn reproduce baseline (đã giải thích trong tài liệu Hướng dẫn Thực nghiệm Phần 3). Tại Giai đoạn 4 này, em chỉ cần adapt module đó cho CatSA.

Logic evaluation: với mỗi phiên test, mô hình tính z_s , tính scores cho toàn vocabulary, sort descending để được ranking. So sánh ranking với ground-truth item, tính $HR@K$, $NDCG@K$, $MRR@K$ cho $K = 10$ và $K = 20$. Tổng hợp trên toàn test set, trả về metrics dictionary.

Lưu ý quan trọng: evaluation phải dùng full-ranking (tính scores cho TOÀN BỘ vocabulary, không sampled negatives) — như đã giải thích trong tài liệu trước. Code-wise: argsort trên $|I|$ items, lấy top-K, kiểm tra target có trong top-K không.

4.5. Hyperparameter mặc định ban đầu

Cho lần chạy đầu tiên, em dùng bộ hyperparameter mặc định an toàn — không tinh chỉnh sớm, để xem mô hình có chạy đúng không trước. Sau khi confirm pipeline OK, mới grid search.

HYPERPARAMETER MẶC ĐỊNH CHO LẦN CHẠY ĐẦU TIÊN:

```

Embedding dimension d      = 100
Số lớp message passing L  = 2
Batch size                 = 100
Learning rate              = 1e-3
Weight decay               = 1e-5
Optimizer                  = Adam
Gradient clipping max_norm = 5.0
Số epoch tối đa            = 30
Early stopping patience    = 5
Random seed (lần đầu)     = 42

```

Bộ này lấy theo SR-GNN gốc — proven hợp lý cho SBR.

Sau khi mô hình chạy đúng, grid search:

```

d ∈ {64, 100, 128}
L ∈ {1, 2, 3}
lr ∈ {1e-4, 5e-4, 1e-3, 5e-3}

```

4.6. Sanity check cuối Giai đoạn 4 — quan trọng nhất

CHECK 1 — Training loss giảm theo epoch:

Sau 5 epoch đầu, loss giảm rõ rệt (ví dụ từ 8.5 xuống 6.2).
Nếu loss không giảm hoặc dao động loạn xạ → có bug.

CHECK 2 — Validation $HR@20$ cải thiện:

Sau 10 epoch, val $HR@20$ phải đạt ít nhất 25% trên Yoochoose (random baseline $\approx 1/|I|$). Nếu $HR@20 \approx 1/|I|$, mô hình

không học được gì.

CHECK 3 – Test HR@20 hợp lý so với SR-GNN:

Em đã reproduce SR-GNN ở giai đoạn baseline. Mô hình A2

(Module 1 only) phải đạt CAO HƠN HOẶC NGANG SR-GNN.

Lý do: A2 = SR-GNN + category nodes, nên không nên kém hơn.

Nếu A2 thấp hơn SR-GNN > 1%, có bug ở Module 1.

CHECK 4 – Overfitting check:

Train loss tiếp tục giảm nhưng val HR@20 đứng yên hoặc giảm

sau 10-15 epoch → bình thường, đó là lý do có early stopping.

Nếu train loss cao mà val cũng cao → có thể underfit, tăng L hoặc d.

CHECK 5 – Reproducibility:

Chạy 2 lần với cùng seed, kết quả phải khớp đến 4 chữ số

thập phân. Nếu không khớp, có nguồn randomness chưa kiểm soát (DataLoader shuffle, dropout, ...).

Gate Check Giai đoạn 4 — CỘT MỐC AN TOÀN: (a) Pipeline end-to-end chạy được trên ít nhất 1 dataset (RetailRocket khuyến nghị làm trước). (b) Năm check trên pass. (c) Mô hình A2 đạt HR@20 ngang hoặc tốt hơn SR-GNN baseline. (d) Toàn bộ pipeline chạy < 4 giờ cho 1 lần train trên 1 dataset. Khi pass, commit Git với message "Stage 4 complete: A2 (Module 1 only) end-to-end". Tại đây em đã có một KẾT QUẢ ĐỂ BÁO CÁO trong bảng ablation. Nếu Giai đoạn 5 hay 6 sau đây gặp vấn đề lớn, em vẫn có A2 làm fallback.

GIAI ĐOẠN 5 — MODULE 2: CATEGORY-GUIDED AUGMENTATION

Mục tiêu giai đoạn này: viết hàm `augment(s)` nhận một phiên gốc và trả về một phiên biến thể có ngữ nghĩa. Module 2 chạy hoàn toàn ở mức input data — nhận phiên, biến đổi phiên, trả về phiên — không động đến encoder hay loss. Nhờ vậy Module 2 tương đối độc lập với Module 1.

Em sẽ implement ba chiến lược augmentation: same-leaf substitution, sibling-leaf substitution, hybrid. Tôi khuyến nghị em làm theo thứ tự này — same-leaf đơn giản nhất, làm trước; sibling-leaf phụ thuộc vào `cat_parent` (chỉ áp dụng được với taxonomy datasets); hybrid kết hợp same-leaf với crop, làm cuối cùng.

5.1. Same-leaf substitution

Đây là phép augmentation đơn giản nhất. Cho phiên $s = [i_1, \dots, i_n]$, chọn ngẫu nhiên k items để substitute ($k = \text{round}(\eta_{\text{aug}} \cdot n)$, với $\eta_{\text{aug}} \approx 0.3$ mặc định). Với mỗi item i_j được chọn, lấy ngẫu nhiên một item $i' \neq i_j$ từ `cat2items[cat(i_j)]` làm thay thế.

PSEUDOCODE — SAME-LEAF SUBSTITUTION:

```
def same_leaf_augment(session, item2cat, cat2items, eta_aug=0.3, k_min=5):
    n = len(session)
    k = max(1, round(eta_aug * n))    # số item cần substitute

    # Chọn k positions ngẫu nhiên
    positions = random.sample(range(n), k)

    augmented = list(session)    # copy
    for pos in positions:
        i_j = session[pos]
        cat_j = item2cat[i_j]
        candidates = cat2items[cat_j]

        # Cơ chế fallback k_min:
        if len(candidates) < k_min:
            # Category quá nhỏ — không substitute, giữ nguyên i_j
            continue

        # Loại bỏ chính i_j khỏi candidates
        valid_candidates = [c for c in candidates if c != i_j]
        if not valid_candidates:
            continue

        # Lấy ngẫu nhiên một item thay thế
        augmented[pos] = random.choice(valid_candidates)

    return augmented
```

Ví dụ:

```
session = [1234, 1235, 1236, 1237]    # iPhone, Ốp lưng, Sạc, AirPods
eta_aug = 0.3 → k = max(1, round(0.3 * 4)) = 1
positions = [0]    # ngẫu nhiên chọn vị trí 0 (iPhone)
cat_j = "Điện thoại"
candidates = [1234, 5678, 9012]    # các điện thoại
valid = [5678, 9012]    # loại 1234 (chính iPhone)
replacement = 5678    # Samsung S25
augmented = [5678, 1235, 1236, 1237]    # Samsung thay iPhone
```

Lưu ý cơ chế fallback: nếu cat quá nhỏ ($|\text{candidates}| < k_{\text{min}}$), em không substitute item đó, giữ nguyên. Đây là quyết định thiết kế đã thảo luận trong tài liệu walkthrough Phần 4.5. Tham số $k_{\text{min}} = 5$ là mặc định, em sẽ tinh chỉnh trong sensitivity analysis ở tháng T7.

5.2. Sibling-leaf substitution

Sibling-leaf phức tạp hơn một chút vì cần hai bước: (a) tìm sibling categories của $\text{cat}(i_j)$, (b) chọn ngẫu nhiên một item từ một sibling category. Yêu cầu cat_parent dictionary — chỉ áp dụng cho RetailRocket và Tmall.

Để tránh tính siblings nhiều lần (rất chậm), em precompute siblings table một lần ở đầu chương trình:

```
PRECOMPUTE SIBLINGS TABLE (LÀM 1 LẦN, REUSE):
```

```
def compute_siblings(cat_parent):
    # parent_to_children: từ parent → list các child categories
    parent_to_children = defaultdict(list)
    for cat, parent in cat_parent.items():
        parent_to_children[parent].append(cat)

    # siblings: từ category → list siblings (cùng parent, khác chính nó)
    siblings = {}
    for cat, parent in cat_parent.items():
        all_children = parent_to_children[parent]
        siblings[cat] = [c for c in all_children if c != cat]

    return siblings
```

```
PSEUDOCODE – SIBLING-LEAF SUBSTITUTION:
```

```
def sibling_leaf_augment(session, item2cat, cat2items,
                        siblings_table, eta_aug=0.3, k_min=5):
    n = len(session)
    k = max(1, round(eta_aug * n))
    positions = random.sample(range(n), k)

    augmented = list(session)
    for pos in positions:
        i_j = session[pos]
        cat_j = item2cat[i_j]
        sibling_cats = siblings_table.get(cat_j, [])

        if not sibling_cats:
            # Không có sibling → fallback sang same-leaf
            candidates = cat2items[cat_j]
            valid = [c for c in candidates if c != i_j]
            if len(valid) < k_min:
                continue
            augmented[pos] = random.choice(valid)
        else:
            # Chọn ngẫu nhiên một sibling category
            sib_cat = random.choice(sibling_cats)
            sib_candidates = cat2items.get(sib_cat, [])
            if len(sib_candidates) < k_min:
                continue
            augmented[pos] = random.choice(sib_candidates)

    return augmented
```

5.3. Hybrid augmentation

Hybrid kết hợp same-leaf substitution với random crop. Logic: trước hết áp dụng same-leaf (giữ intent), sau đó áp dụng crop (tăng đa dạng cấu trúc). Em có thể thử cả sibling + crop hoặc same + mask, nhưng khuyến nghị bắt đầu với same + crop là cấu hình phổ biến nhất.

PSEUDOCODE — HYBRID AUGMENTATION:

```
def hybrid_augment(session, item2cat, cat2items, eta_aug=0.3,
                    eta_crop=0.75, k_min=5):
    # Bước 1: same-leaf substitution
    augmented = same_leaf_augment(session, item2cat, cat2items,
                                   eta_aug=eta_aug, k_min=k_min)

    # Bước 2: random crop
    n = len(augmented)
    crop_len = max(1, int(n * eta_crop))
    start = random.randint(0, n - crop_len)
    augmented = augmented[start : start + crop_len]

    return augmented
```

Ví dụ:

```
session = [1234, 1235, 1236, 1237]
Sau same-leaf: [5678, 1235, 1236, 1237]    # Samsung thay iPhone
eta_crop = 0.75 → crop_len = 3
start = random.randint(0, 1) = 0
Kết quả: [5678, 1235, 1236]    # mất AirPods do crop
```

5.4. Hàm augment chung — chọn ngẫu nhiên một strategy

Trong CatSA, ba strategy không hoạt động riêng lẻ mà được chọn ngẫu nhiên cho mỗi cặp dương. Lý do: tăng đa dạng tín hiệu học. Mỗi cặp (s, ã) có thể được sinh bởi same-leaf, sibling-leaf, hoặc hybrid với xác suất bằng nhau (hoặc có weight tinh chỉnh được).

HÀM AUGMENT CHUNG TRONG CATSA:

```
def catsa_augment(session, item2cat, cat2items, siblings_table=None,
                  strategies=['same', 'sibling', 'hybrid'],
                  strategy_weights=None, eta_aug=0.3, k_min=5):

    # Nếu dataset không có taxonomy, loại bỏ 'sibling'
    if siblings_table is None:
        strategies = [s for s in strategies if s != 'sibling']

    # Chọn ngẫu nhiên 1 strategy
    if strategy_weights:
        strategy = random.choices(strategies, weights=strategy_weights)[0]
    else:
        strategy = random.choice(strategies)

    if strategy == 'same':
        return same_leaf_augment(session, item2cat, cat2items,
                                   eta_aug=eta_aug, k_min=k_min)
    elif strategy == 'sibling':
        return sibling_leaf_augment(session, item2cat, cat2items,
                                    siblings_table, eta_aug=eta_aug,
                                    k_min=k_min)
```

```
elif strategy == 'hybrid':  
    return hybrid_augment(session, item2cat, cat2items,  
                           eta_aug=eta_aug, k_min=k_min)
```

5.5. Sanity check cuối Giai đoạn 5

CHECK 1 – Mỗi strategy chạy không lỗi trên 1000 phiên:
Lấy 1000 phiên từ training set, áp mỗi strategy lần lượt.
Không có exception, không có session rỗng (len = 0).

CHECK 2 – Augmented session khác phiên gốc:
Tính tỷ lệ phiên có augmented[i] == original[i] cho tất cả i.
Mục tiêu: chỉ < 5% trường hợp giống y nguyên (do fallback k_min).
Nếu > 50% giống y nguyên, eta_aug quá thấp hoặc nhiều cat quá nhỏ.

CHECK 3 – Augmented session preserves intent (qualitative):
Lấy 5-10 phiên ngẫu nhiên, in ra phiên gốc và augmented.
Em đọc và xác nhận: ý định mua hàng còn giữ?
Ví dụ phiên gốc về điện thoại + phụ kiện, augmented vẫn là
điện thoại + phụ kiện (chỉ khác hãng, hoặc loại phụ kiện).

CHECK 4 – Distribution of strategies (khi dùng hàm chung):
Augment 10000 phiên với ba strategy đều xác suất.
Đếm số lần mỗi strategy được chọn.
Mục tiêu: ba strategy đều ~33% (sai số nhỏ).

CHECK 5 – Speed:
10000 phiên augment trong < 5 giây trên CPU (không cần GPU).
Module 2 chạy trên CPU vì là pure Python random sampling.

Gate Check Giai đoạn 5: (a) Ba strategy implement đầy đủ và pass năm check. (b) Hàm catsa_augment chung hoạt động đúng. (c) Em đã visualize ít nhất 5 cặp (gốc, augmented) và xác nhận intent được giữ. (d) Speed test pass (< 5s cho 10000 phiên). Khi pass, commit Git với message "Stage 5 complete: Module 2 augmentation".

GIAI ĐOẠN 6 — SESSION-LEVEL INFO NCE VÀ TÍCH HỢP CUỐI CÙNG

Đây là giai đoạn cuối cùng để có CatSA đầy đủ. Em sẽ làm ba việc kế tiếp nhau theo trình tự logic. Việc thứ nhất là viết hàm tính session-level InfoNCE loss. Việc thứ hai là tích hợp Module 2 (augmentation) vào training loop của Giai đoạn 4 — sao cho mỗi batch tự sinh các phiên biến thể. Việc thứ ba là kết hợp L_{rec} và $L_{\text{CL-session}}$ thành multi-task objective. Khi xong việc thứ ba, em đã có CatSA đầy đủ và có thể chạy thí nghiệm trong tháng T6.

6.1. Hàm tính session-level InfoNCE loss

Em đã hiểu công thức InfoNCE qua tài liệu walkthrough Phần 5.2. Trong Giai đoạn 6 này, em viết hàm cụ thể tính loss này từ một batch các session embeddings. Logic không phức tạp nhưng có một vài chi tiết cần chú ý.

Đầu vào của hàm: hai tensor z_s _batch và z_s _tilde_batch, cả hai đều có shape (B, d), với B là batch size và d là embedding dimension. z_s _batch[i] là embedding của phiên gốc thứ i, z_s _tilde_batch[i] là embedding của phiên biến thể tương ứng. Đầu ra: một scalar loss.

PSEUDOCODE — SESSION-LEVEL InfoNCE LOSS:

```
import torch
import torch.nn.functional as F

def session_level_infonce(z_s, z_s_tilde, tau=0.5):
    """
    z_s:          tensor shape (B, d) — embedding của phiên gốc
    z_s_tilde:    tensor shape (B, d) — embedding của phiên augmented
    tau:          temperature (mặc định 0.5)
    """
    B = z_s.size(0)

    # Bước 1: chuẩn hoá L2 (cosine similarity = dot product sau khi
    normalize)
    z_s = F.normalize(z_s, dim=1)          # (B, d)
    z_s_tilde = F.normalize(z_s_tilde, dim=1)  # (B, d)

    # Bước 2: tính similarity matrix giữa z_s và z_s_tilde
    # sim[i, j] = sim(z_s[i], z_s_tilde[j])
    sim = torch.mm(z_s, z_s_tilde.T)      # (B, B)

    # Bước 3: chia cho temperature
    sim = sim / tau

    # Bước 4: target = đường chéo
    # Cặp dương là (z_s[i], z_s_tilde[i]) → ở vị trí sim[i, i]
    # InfoNCE = cross-entropy với target là index i
    targets = torch.arange(B, device=z_s.device)

    # Cross-entropy theo HÀNG
    loss_row = F.cross_entropy(sim, targets)

    # Cross-entropy theo CỘT (đối xứng — cải thiện ổn định)
    loss_col = F.cross_entropy(sim.T, targets)

    loss = (loss_row + loss_col) / 2
```

```
return loss
```

Diễn giải từng bước:

Bước 1 (normalize): chuyển dot product thành cosine similarity.
Đảm bảo similarity $\in [-1, 1]$, không bị ảnh hưởng bởi độ dài vector.

Bước 2 (sim matrix): tính similarity giữa mọi cặp $(z_s[i], z_s_tilde[j])$.
Đường chéo $sim[i,i]$ là cặp dương (i với i).
Off-diagonal $sim[i,j]$ ($i \neq j$) là cặp âm.

Bước 3 (temperature): tau nhỏ làm sim sắc nét, gradient mạnh hơn.

Bước 4 (cross-entropy): InfoNCE chính là cross-entropy với target là vị trí cặp dương trong row (hoặc cột). PyTorch có sẵn `F.cross_entropy`.

Một chi tiết quan trọng: tính loss theo cả hàng và cột (symmetric InfoNCE), rồi lấy trung bình. Cách này được dùng trong nhiều paper CL hiện đại (như SimCLR) vì cho gradient ổn định hơn và đối xứng giữa $(s, \tilde{s})$ và $(\tilde{s}, s)$.

6.2. Ví dụ minh họa — InfoNCE trên batch nhỏ

Để em hình dung cụ thể, hãy tính InfoNCE trên một batch $B = 3$ với $d = 4$ (chiều thấp cho dễ theo dõi).

VÍ DỤ — InfoNCE TRÊN BATCH $B=3$, $d=4$:

```
z_s = [[0.6, -0.2, 0.5, 0.4],      # phiên gốc 1
        [0.3,  0.4, -0.1, 0.7],    # phiên gốc 2
        [-0.5, 0.6, 0.4, 0.2]]     # phiên gốc 3

z_s_tilde = [[0.55, -0.18, 0.52, 0.38], # augmented 1 (gần z_s[0])
              [0.28,  0.42, -0.05, 0.68], # augmented 2 (gần z_s[1])
              [-0.48, 0.58, 0.38, 0.22]]  # augmented 3 (gần z_s[2])
```

Sau normalize L2, similarity matrix:

```
      ↓ cột: phiên augmented thứ j
hàng: phiên gốc thứ i
      j=0      j=1      j=2
i=0 [  0.99   -0.05   -0.32 ]
i=1 [ -0.04    0.99   -0.10 ] ← đường chéo SHOULD lớn
i=2 [ -0.30   -0.08    0.99 ]   (cặp dương)
```

Đường chéo: 0.99, 0.99, 0.99 → đúng cặp dương là gần nhau.

Sau khi chia cho tau = 0.5:

```
sim/tau = [[1.98, -0.10, -0.64],
           [-0.08, 1.98, -0.20],
           [-0.60, -0.16, 1.98]]
```

Áp softmax theo từng hàng (từ số là giá trị đường chéo):

```
row 0: exp(1.98) / (exp(1.98) + exp(-0.10) + exp(-0.64))
      = 7.24 / (7.24 + 0.90 + 0.53)
      = 7.24 / 8.67 ≈ 0.835
→ -log(0.835) ≈ 0.180
```


Tương tự rows 1, 2 $\rightarrow$ loss row ≈ 0.180 cho mỗi row
loss_row = trung bình ≈ 0.180

loss_col tính tương tự ≈ 0.180

Final loss = $(0.180 + 0.180) / 2 = 0.180$

Diễn giải:

- Loss thấp (0.18) vì các cặp dương ở đường chéo có sim rất cao (0.99) so với các cặp âm (gần 0). Mô hình đã học tốt cho batch này.
- Khi mới khởi tạo, embedding ngẫu nhiên $\rightarrow$ đường chéo không cao hơn off-diagonal $\rightarrow$ loss $\approx \log(B) = \log(3) \approx 1.10$.
- Sau training, loss giảm dần khi mô hình học phân biệt cặp dương/âm.

6.3. Tích hợp Module 2 vào training loop

Đến đây em đã có hàm augment (Giai đoạn 5) và hàm session_level_infonce (mục 6.1). Việc còn lại là cập nhật training loop của Giai đoạn 4 để sinh phiên augmented và tính cả hai loss.

PSEUDOCODE – TRAINING LOOP CÓ AUGMENTATION VÀ CL:

```
for epoch in range(n_epochs):
    model.train()
    total_loss = 0.0
    total_loss_rec = 0.0
    total_loss_cl = 0.0

    for batch in train_loader:
        # Trong DataLoader, mỗi item trả về (graph_orig, target).
        # Ở đây ta cần thêm graph_aug từ session augmented.

        graphs_orig, targets = batch
        graphs_orig = graphs_orig.to(device)
        targets = targets.to(device)

        # — Phần MỚI: build augmented graphs —
        sessions_orig = batch_to_sessions(graphs_orig) # decode
        sessions_aug = [
            catsa_augment(s, item2cat, cat2items, siblings_table)
            for s in sessions_orig
        ]
        graphs_aug = batch_from_sessions(sessions_aug)
        graphs_aug = graphs_aug.to(device)

        # — Forward pass cho cả hai version —
        z_s = model(graphs_orig) # (B, d)
        z_s_tilde = model(graphs_aug) # (B, d)

        # — L_rec (chỉ dùng phiên gốc) —
        item_emb_table = model.item_emb.weight # (|I|, d)
        logits = z_s @ item_emb_table.T # (B, |I|)
        loss_rec = F.cross_entropy(logits, targets)

        # — L_CL-session —
        loss_cl = session_level_infonce(z_s, z_s_tilde, tau=0.5)

        # — Multi-task loss —
        loss = loss_rec + lam * loss_cl
```

```

# Backward + update
optimizer.zero_grad()
loss.backward()
torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=5.0)
optimizer.step()

total_loss += loss.item() * targets.size(0)
total_loss_rec += loss_rec.item() * targets.size(0)
total_loss_cl += loss_cl.item() * targets.size(0)

# Log cả ba loss để dễ debug
n = len(train_loader.dataset)
print(f"Epoch {epoch}: total={total_loss/n:.4f}, "
      f"rec={total_loss_rec/n:.4f}, cl={total_loss_cl/n:.4f}")

```

Lưu ý quan trọng về implementation: hàm `batch_to_sessions` và `batch_from_sessions` là utility helpers em phải viết để chuyển đổi giữa session list và HeteroData batch. Cụ thể, `batch_to_sessions` decode batch PyG ngược về list of session lists; `batch_from_sessions` làm ngược lại. Một cách thay thế đơn giản hơn: pass cả `sessions_orig` và `sessions_aug` từ `DataLoader` (modify `Dataset` class), tránh decode-encode.

6.4. Lựa chọn hyperparameter cho CL

Trong CatSA đầy đủ, em có hai hyperparameter mới so với Giai đoạn 4: λ (trọng số CL) và τ (temperature). Cộng với η_{aug} và k_{min} từ Module 2, tổng cộng có bốn hyperparameter cần tinh chỉnh.

HYPERPARAMETER MẶC ĐỊNH CHO CATSA ĐẦY ĐỦ:

Tất cả hyperparameter của Giai đoạn 4, cộng:

λ (CL weight)	= 0.1	(sweep: 0.001, 0.01, 0.1, 0.5, 1.0)
τ (temperature)	= 0.5	(sweep: 0.05, 0.1, 0.2, 0.5, 1.0)
η_{aug} (substitute rate)	= 0.3	(sweep: 0.2, 0.3, 0.5, 0.7)
k_{min} (fallback thresh.)	= 5	(sweep: 3, 5, 10, 20)

Khuyến nghị thứ tự tuning:

1. Lấy mặc định trên tất cả, train 1 lần để confirm pipeline OK (gate check Giai đoạn 6).
2. Tuning λ trước (ảnh hưởng lớn nhất). Cố định các hp khác.
3. Tuning τ . Cố định λ tốt nhất từ bước 2.
4. Tuning η_{aug} . Thường ít ảnh hưởng, có thể giữ mặc định.
5. Tuning k_{min} trên dataset có nhiều cat nhỏ (RetailRocket, Tmall).

Cách tuning: grid search trên validation set cho mỗi dataset.
 Báo cáo trong Bảng Sensitivity Analysis (T7).

6.5. Sanity check cuối Giai đoạn 6

CHECK 1 – Hai loss cùng giảm theo epoch:
 Cả `total_loss`, `loss_rec`, `loss_cl` phải có xu hướng giảm.
 Nếu `loss_cl` tăng hoặc dao động loạn xạ trong khi `loss_rec` giảm → λ quá lớn hoặc τ chưa phù hợp.

CHECK 2 – CatSA đầy đủ tốt hơn A2 (Giai đoạn 4):

So sánh val HR@20 của CatSA đầy đủ với A2.

Mục tiêu: CatSA cải thiện ít nhất 0.5% so với A2.

Nếu CatSA = A2, Module 2 + CL không có giá trị bổ sung

→ quay lại debug.

CHECK 3 – CatSA tốt hơn CL4SRec (baseline trực tiếp nhất):

So sánh CatSA với CL4SRec đã reproduce ở giai đoạn baseline.

Mục tiêu: CatSA cải thiện ít nhất 0.5% trên ≥ 1 dataset.

Nếu CatSA = CL4SRec, category-guided augmentation không tốt hơn random augmentation → cần phân tích sâu (có thể dataset không phù hợp, hoặc bug trong augmentation).

CHECK 4 – Reproducibility với 5 seed:

Chạy với 5 seed trên 1 dataset, kiểm tra std < 0.5% HR@20.

Std lớn hơn → mô hình không ổn định, có vấn đề.

CHECK 5 – Speed:

Mỗi epoch CatSA mất < 2x thời gian của A2.

Lý do: CatSA forward pass 2 lần (z_s và z_{s_tilde}) + augmentation overhead.

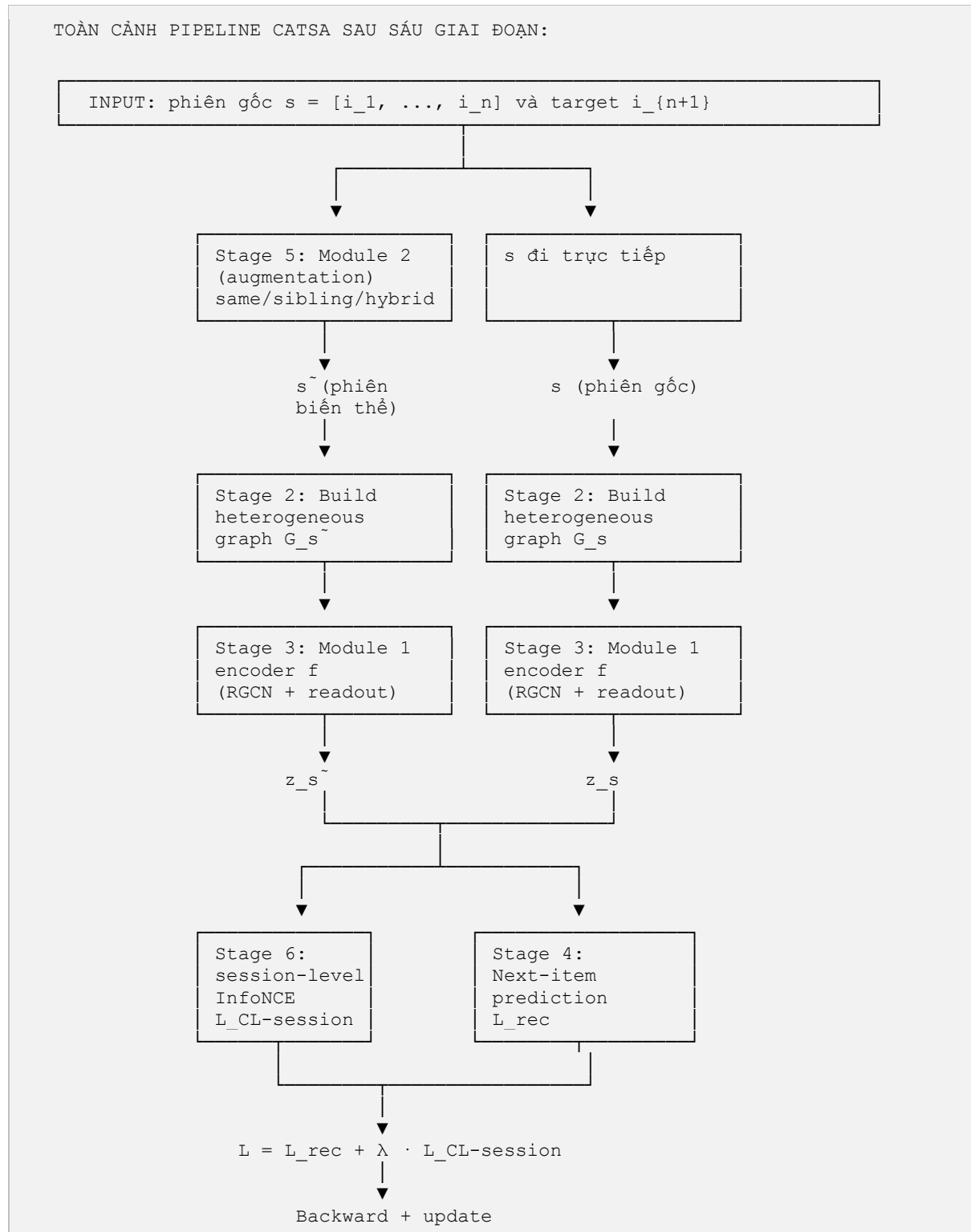
Nếu chậm hơn 2x, kiểm tra augmentation có bị bottleneck.

Gate Check Giai đoạn 6 — CỘT MỐC HOÀN THÀNH CATSA: (a) CatSA đầy đủ chạy end-to-end với cả ba thành phần (Module 1, Module 2, session-CL). (b) Năm check trên pass — đặc biệt CHECK 2 (vượt A2) và CHECK 3 (vượt CL4SRec). (c) Em có thể demo cho Thầy: chạy CatSA trên RetailRocket trong 1 buổi, cho ra kết quả. (d) Code đã clean, có docstring, có config file, sẵn sàng cho thí nghiệm chính ở T6. Khi pass, commit Git với message "Stage 6 complete: CatSA full pipeline". CHÚC MỪNG! Em đã hoàn thành phần khó nhất của bài Q3.

TỔNG KẾT — TRỊ TUỆ TỪ SÁU GIAI ĐOẠN

Em đã đi qua sáu giai đoạn xây dựng CatSA. Phần này tổng kết các bài học chính và đặt sáu giai đoạn vào bức tranh tổng thể của lộ trình thí nghiệm. Đọc kỹ phần này trước khi bắt đầu — nó giúp em không bị lạc hướng giữa các chi tiết kỹ thuật.

Sơ đồ tích hợp toàn bộ CatSA



Bốn nguyên tắc chiến lược em rút ra

Nhìn lại sáu giai đoạn, có bốn nguyên tắc làm việc mà tôi muốn em mang theo trong toàn bộ luận án, không chỉ cho bài Q3.

Nguyên tắc thứ nhất là scaffolding — xây từng phần nhỏ trên nền phần đã chạy ổn định. Em đã thấy: tại cuối Giai đoạn 4, em đã có A2 hoạt động — đây là điểm an toàn. Nếu Giai đoạn 5 hay 6 gặp vấn đề lớn, em không trắng tay. Áp dụng nguyên tắc này cho cả các bài Q2 và Q1: luôn có một version trung gian chạy được trước khi thêm thành phần phức tạp.

Nguyên tắc thứ hai là sanity check trước khi đi tiếp. Mỗi giai đoạn có 4-5 check ngắn. Đừng skip — 5 phút check ngay tại đó tiết kiệm 5 giờ debug về sau. Đặc biệt các check về speed và shape — bug shape tensor (1, d) vs (d,) có thể gây sai số ngầm rất khó phát hiện.

Nguyên tắc thứ ba là visualize và print đủ. Trong Giai đoạn 1 em vẽ phân bố category. Trong Giai đoạn 2 em vẽ heterogeneous graph. Trong Giai đoạn 5 em đọc các cặp (gốc, augmented). Mỗi lần visualize, em phát hiện được những điều mà chỉ nhìn số không thấy. Cuối tháng T7 khi viết phân tích kết quả, em sẽ cần t-SNE, sensitivity plots — đầu tư thói quen visualize từ sớm.

Nguyên tắc thứ tư là báo Thầy ngay khi không tự debug được sau 3 ngày. Nhiều khi vấn đề là một dòng code đơn giản, nhìn từ một góc khác sẽ thấy ngay. Đừng cố gắng "tự lực" quá lâu — em vẫn cần học hỏi phương pháp luận debugging từ Thầy, không chỉ nội dung khoa học.

Lộ trình thời gian cho sáu giai đoạn

Giai đoạn	Tuần	Trạng thái cuối	Có thể demo Thầy?
1: Data prep + lookup	Tuần 1	item2cat, cat2items, cat_parent có cho 4 dataset	Bảng phân bố cat
2: Heterogeneous graph	Tuần 2	Hàm session_to_graph chạy được	Vẽ 1 graph
3: RGCN encoder	Tuần 3-4 (giữa)	Encoder forward pass cho ra z_s đúng shape	Forward pass demo
4: L_rec end-to-end	Tuần 4 (cuối)-5	A2 chạy được, cho $HR@20 \geq SR-GNN$	Bảng kết quả A2
5: Module 2 augmentation	Tuần 6	Ba strategy hoạt động, bảo toàn intent	Cặp gốc/augmented
6: Session-level CL	Tuần 7-8	CatSA đầy đủ, vượt A2 và CL4SRec	Bảng kết quả CatSA

Bảng 1. Lộ trình thời gian cho sáu giai đoạn xây dựng CatSA. Tổng 7-8 tuần, khớp với tháng T4 và đầu T5 trong Roadmap. Mỗi giai đoạn có deliverable cụ thể em có thể demo cho Thầy tại các meeting hàng tuần.

Sau khi hoàn thành sáu giai đoạn

Khi qua được Giai đoạn 6, em đã có CatSA đầy đủ và sẵn sàng cho thí nghiệm chính ở tháng T6. Bước tiếp theo là chạy CatSA trên cả 4 dataset với 5 seed mỗi dataset, lập bảng baselines chính (theo hướng

dẫn trong tài liệu Hướng dẫn Thực nghiệm Phần 4). Sau đó tháng T7 là ablation, sensitivity, sub-population analysis. Tháng T8 là viết paper. Tháng T9 là revision và submit.

— Hết tài liệu —

PGS. TS. Nguyễn Văn Hậu | Tháng 5/2026